

Suggested JSON Schema

This document describes the JSON schema actually enforced by the backend

([backend/app/models/schemas.py](#), Pydantic models) — this is not an aspirational schema, it is the exact contract the LLM output is validated against before any response reaches the frontend.

1. Complete schema (reference form)

```
{
  "client_id": "string",
  "report_id": "string",
  "period": {
    "start_day": "string",
    "end_day": "string",
    "days_covered": ["string"]
  },
  "weekly_summary": { "value": "...", "classification": "...", "confidence": 0.0,
  "evidence": [ { "day": "...", "speaker": "...", "quote": "..."} ] },
  "nutrition": {
    "adherence": "<GradedField>",
    "notes": "<GradedField>"
  },
  "exercise": {
    "steps": "<GradedField>",
    "activity_type": "<GradedField>",
    "consistency": "<GradedField>"
  },
  "sleep": {
    "average_hours": "<GradedField>",
    "quality_notes": "<GradedField>"
  },
  "water": "<GradedField>",
  "symptoms": {
    "reported_symptoms": "<GradedField>",
    "frequency_pattern": "<GradedField>"
  },
  "stress": "<GradedField>",
  "engagement": {
    "level": "<GradedField>",
    "responsiveness": "<GradedField>",
    "missed_checkins": "<GradedField>"
  },
  "key_barriers": [ { "description": "string", "classification": "...",
  "confidence": 0.0, "evidence": [ "..." ] } ],
  "pending_actions": [ { "description": "string", "source":
  "coach|client|accountability_coach", "status": "open|completed|carried_over",
```

```
"classification": "...", "confidence": 0.0, "evidence": [ "..." ] } ],
  "risk_flags": [ { "flag": "string", "severity": "low|medium|high",
"rule_triggered": "string", "classification": "...", "confidence": 0.0,
"evidence": [ "min 1 item" ] } ],
  "coach_recommendations": [ { "recommendation": "string", "priority":
"low|medium|high", "classification": "...", "confidence": 0.0, "evidence": [ "...
" ] } ],
  "conflicting_reports": [ { "description": "string", "classification":
"conflicting_reports", "evidence": [ "min 2 items" ] } ],
  "missing_information": [ { "category": "string", "note": "string" } ]
}
```

Where `<GradedField>` is the repeated building block:

```
{
  "value": "string | number | boolean | array | null",
  "classification": "confirmed_fact | client_reported | ai_inference |
missing_information | conflicting_reports",
  "confidence": 0.0,
  "evidence": [
    { "day": "string", "speaker": "string (optional)", "quote": "string, max 300
chars" }
  ]
}
```

Full HTTP response envelope (`AnalyzeResponse`):

```
{
  "success": true,
  "data": "<ClientIntelligenceReport, as above, present only on success>",
  "error": null
}
```

Note on `error`: the field exists on the response model for forward compatibility, but the current implementation does not populate it — failures are communicated via HTTP status code (502/500) plus a `detail` message in the FastAPI error response, not via a 200 with `success: false`. This is documented explicitly so the schema matches actual behavior rather than an aspirational design.

2. Explanation of major fields

Top level

Field	Purpose
<code>client_id</code> , <code>report_id</code>	Identifiers. Deliberately not trusted from the LLM — <code>analysis_service.py</code> backfills these deterministically after parsing, since an identifier is not an analytical finding and shouldn't be subject to model variance.

Field	Purpose
period	The day range covered, plus the explicit list of days present in the transcript. Anchors every other field to a concrete time window rather than a vague "this week."
weekly_summary	A single GradedField — a short narrative synthesis, still required to carry evidence and a classification like every other finding, so it can't become a place for the model to make claims it can't back up elsewhere in the report.

The GradedField pattern

This is the core design decision in the schema (see §3). Every individual metric — nutrition adherence, step count, sleep hours, water intake, symptom pattern, stress, each engagement dimension — is a GradedField with exactly four keys:

- **value** — the extracted finding itself, in the model's own words or as a number. `null`/empty only makes sense when `classification` is `missing_information`.
- **classification** — which of the five categories the finding belongs to (see below).
- **confidence** — `0.0–1.0`, how directly the transcript supports this specific claim.
- **evidence** — an array of `{day, speaker, quote}` objects. The schema enforces (via a custom validator) that this array cannot be empty unless `classification` is `missing_information` — a claim with no evidence simply cannot pass validation.

Classification enum (five values)

Value	Meaning
confirmed_fact	Stated as objective fact by a coach/accountability coach, or a structured numeric log entry.
client_reported	The client's own self-report — a real thing they said, but not independently verified.
ai_inference	A pattern or judgment the model derived by connecting two or more data points. Must cite everything it was derived from.
missing_information	The category was not addressed anywhere in the transcript.
conflicting_reports	Two or more statements contradict each other; both sides are reported rather than one being silently discarded.

Category groupings (nutrition, exercise, sleep, symptoms, engagement)

Each groups 2–3 related GradedFields under one object (e.g. `exercise.steps`, `exercise.activity_type`, `exercise.consistency`). `water` and `stress` are single GradedFields at the top level since they didn't need sub-breakdown for this transcript format.

List-based sections

Field	Extra fields beyond the <code>GradedField</code> base	Notes
<code>key_barriers</code>	<code>description</code>	Recurring obstacles the client faces.
<code>pending_actions</code>	<code>description</code> , <code>source</code> (coach/client/accountability_coach), <code>status</code> (open/completed/carried_over)	Tracks commitments made in the conversation.
<code>risk_flags</code>	<code>flag</code> , <code>severity</code> (low/medium/high), <code>rule_triggered</code>	Minimum 1 evidence item enforced by the schema. <code>rule_triggered</code> requires the model to state the explicit threshold that justified the flag (e.g. "sleep < 6h on 3+ of the last 7 days"), rather than raising a flag on tone alone.
<code>coach_recommendations</code>	<code>recommendation</code> , <code>priority</code> (low/medium/high)	Suggested next actions for the coach, still evidence-linked.
<code>conflicting_reports</code>	<code>description</code>	Minimum 2 evidence items enforced by the schema — a conflict by definition needs two conflicting statements shown side by side.
<code>missing_information</code>	<code>category</code> , <code>note</code>	Explicit "no data" entries — every relevant category should be represented here if the transcript is silent on it, so an absence is a stated fact, not a silent omission.

3. Why this schema was chosen

- **A single reusable `GradedField` unit** rather than one bespoke shape per category keeps the whole report internally consistent — the frontend has exactly one card component (`GradedFieldCard.jsx`) that renders every metric identically, and every metric in the report is auditable the same way.
- **Evidence is structural, not optional.** By encoding "no evidence $\Rightarrow$ invalid unless missing" as a Pydantic validator rather than a prompt instruction, grounding is enforced by code that cannot be talked out of it by a persuasive but unsupported LLM response.
- **A fixed five-value classification enum**, rather than free-text labels, means the frontend can map classification directly to a consistent color/badge system (`utils/classification.js`) and the backend can validate it with a plain enum check — no fuzzy string matching required anywhere in the pipeline.
- **Minimum evidence counts on `risk_flags` (1) and `conflicting_reports` (2)** reflect that these are the two highest- stakes, highest-scrutiny output types — a risk flag with zero evidence, or a "conflict"

backed by only one quote, is not just weak, it's incoherent, so the schema refuses to accept it rather than relying on the model to remember the rule.

- **missing_information as a first-class, always-populatable list** removes the incentive for the model to fill silence with a plausible-sounding guess — saying "no data" is a fully valid answer the schema expects, not an omission to be avoided.
- **Deterministic backfill of report_id/client_id** keeps identifiers out of the LLM's hands entirely, since these need to be stable and code-generated, not subject to model phrasing variance.